

NextEra Energy

Equipment Health Analysis using CockroachDB

Recommended approach

For the first NextEra use case, CRDB should be the application data and knowledge layer for Equipment Health Analysis.

CRDB would store:

1. Plants and equipment.
2. Fault events.
3. Selected health measurements and calculated summaries.
4. Document names, versions, and source locations.
5. Extracted document sections.
6. Vector embeddings for those sections.
7. Relationships between equipment, faults, and documents.
8. Document access rules.
9. Diagnostic results.
10. The evidence and citations used in each answer.

CRDB would not replace the Historian, SharePoint, Confluence, or object storage.

Raw telemetry remains in the Historian system. Original PDFs, Word files, images, and engineering reports remain in the document system or object storage.

The initial design does not require JSONB, GIN indexes, trigram indexes, or full text indexes.

We need to chunk the documents and store embeddings in CockroachDB

For this design, we should divide the approved document content into smaller sections, generate an embedding for each section, and store both the extracted text and its embedding in CRDB.

The embedding is generated outside CRDB by the selected embedding model. CRDB stores the resulting vector and uses it for similarity search.

Each stored section would include:

1. The extracted text.
2. Its vector embedding.
3. The source document and version.
4. The page number.
5. The section heading.
6. The document type.
7. The retrieval scope.
8. The embedding model and version.

This is important because the vector finds relevant content, but the extracted text is what the application gives to the language model as evidence.

The vector alone is not useful to the language model. It is only a mathematical representation used to find the most relevant sections.

Here is an example:

A 200 page inverter manual remains as a PDF in SharePoint.

The document processing service might divide it into sections such as:

1. Safety warnings.
2. Fault code B17.
3. Derating conditions.
4. Temperature operating limits.
5. Reset procedure.
6. Escalation procedure.

Each section becomes a row in CRDB with its text, page number, section heading, and embedding.

We should split documents at meaningful boundaries such as headings, paragraphs, procedure steps, and tables. We should not blindly divide every document into fixed blocks without checking the document structure.

The final chunk size and any overlap should be tested with NextEra's actual manuals. It depends on the embedding model, document structure, and questions users ask.

Why the source URI is still important

The source URI is a pointer to the original document. It is not used for vector search and it does not contain the document content.

It is useful for four reasons.

1. Human verification

A technician can open the original manual or procedure and verify the source behind an answer.

For example, an answer could show:

Source: Inverter Maintenance Manual, version 4.2, page 73

The link would open the approved source document.

2. Auditability

CRDB records which document version and page supported an answer. This allows NextEra to review why the application produced a recommendation.

3. Document refresh

When a document changes, the ingestion service can retrieve the new version, extract it again, create new chunks, and generate new embeddings.

The application should track more than the URI:

```
source_system
source_document_id
version_label
source_uri
source_modified_at
content_hash
```

The source document ID identifies the document. The version and content hash identify the exact content that was processed. The URI provides a way to reach the original.

4. Avoiding document duplication

The original PDF does not need to be copied into CRDB. CRDB stores only the information needed for retrieval, grounding, citations, and application workflow.

The stored URI should be a stable source location or source identifier. We should not store an expiring signed URL. If the document requires authentication, the application should create an authorized link when the user opens the citation.

The URI does not grant access. Permissions must be checked separately.

Storage boundaries

Historian

The Historian continues to store:

1. Raw equipment telemetry.
2. High frequency measurements.
3. Long measurement histories.
4. Original Historian events.
5. Existing Historian calculations.

CRDB should not ingest all second by second telemetry from every plant for this use case.

SharePoint, Confluence, or object storage

The document system continues to store:

1. PDF files.
2. Word files.
3. Images and diagrams.
4. Engineering reports.
5. Original document permissions.
6. Document version history.

CockroachDB

CRDB stores:

1. Structured equipment information.
2. Fault events.
3. Selected Historian results and summaries.
4. Document catalog and version information.
5. Extracted document sections.
6. Vector embeddings.
7. Exact document relationships.
8. Retrieval scopes and permissions.
9. Diagnostic requests and results.
10. Source citations and audit records.

Structured equipment model

```
CREATE TABLE plants (  
    plant_id        UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
    plant_name      STRING NOT NULL,  
    plant_code      STRING NOT NULL UNIQUE,
```

```

        region          STRING,
        timezone        STRING NOT NULL,
        commissioned_at  TIMESTAMPTZ
    );

CREATE TABLE equipment (
    equipment_id        UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    plant_id            UUID NOT NULL REFERENCES plants (plant_id),
    source_equipment_id STRING NOT NULL,
    equipment_type      STRING NOT NULL,
    manufacturer        STRING,
    model               STRING,
    serial_number        STRING,
    firmware_version     STRING,
    commissioned_at      TIMESTAMPTZ,
    operational_status   STRING,
    UNIQUE (plant_id, source_equipment_id)
);

```

```

CREATE INDEX equipment_by_plant
ON equipment (plant_id, equipment_type);

```

These are normal relational columns. We should add another typed column when NextEra has an equipment property that is important for filtering, joining, reporting, or diagnostics.

We should not create a general `equipment_metadata JSONB` column without a specific need.

Fault events

```

CREATE TABLE equipment_fault_events (
    fault_event_id  UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    equipment_id    UUID NOT NULL REFERENCES equipment
(equipment_id),
    source_event_id STRING,
    fault_code      STRING NOT NULL,
    fault_class     STRING,
    severity        STRING,
    controller_state STRING,
    description     STRING,

```

```

        occurred_at      TIMESTAMPTZ NOT NULL,
        cleared_at       TIMESTAMPTZ,
        source_record_uri STRING,
        UNIQUE (equipment_id, source_event_id)
    );

```

```

CREATE INDEX fault_events_by_equipment_time
ON equipment_fault_events (
    equipment_id,
    occurred_at DESC
);

```

```

CREATE INDEX fault_events_by_code
ON equipment_fault_events (
    fault_code,
    occurred_at DESC
);

```

The source record URI points to the original Historian record if deeper investigation is required. The complete Historian response does not need to be copied into JSONB.

Equipment health calculations

The language model should not calculate equipment health from raw telemetry.

A diagnostic service should call the Historian, retrieve the necessary measurements, and perform the engineering calculations using fixed code.

Examples include:

1. Temperature variance.
2. Voltage deviation.
3. Charge or discharge behavior.
4. Fault frequency.
5. Performance compared with similar equipment.
6. Weather correlation.
7. Measurements before and after a fault.

The useful results are then stored as typed rows:

```

CREATE TABLE equipment_metric_summaries (
    metric_summary_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),

```

```

        equipment_id    UUID NOT NULL REFERENCES equipment
(equipment_id),
        metric_name     STRING NOT NULL,
        aggregation_type STRING NOT NULL,
        window_start    TIMESTAMPTZ NOT NULL,
        window_end      TIMESTAMPTZ NOT NULL,
        metric_value     DECIMAL NOT NULL,
        unit            STRING,
        sample_count     INT,
        operating_mode   STRING,
        weather_category STRING,
        firmware_version STRING,
        source_record_uri STRING,
        calculated_at    TIMESTAMPTZ NOT NULL DEFAULT now()
);

```

```

CREATE INDEX metric_summaries_by_equipment
ON equipment_metric_summaries (
    equipment_id,
    metric_name,
    window_end DESC
);

```

There is no JSONB dimensions column. If operating mode, weather category, or firmware version matters, it should have a normal column.

Document catalog

The logical identity of a document is stored separately from its versions.

```

CREATE TABLE documents (
    document_id    UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    source_system   STRING NOT NULL,
    source_document_id STRING NOT NULL,
    title          STRING NOT NULL,
    document_type   STRING NOT NULL,
    discipline      STRING,
    manufacturer    STRING,
    equipment_model  STRING,

```

```
        UNIQUE (source_system, source_document_id)
    );
```

Each version is recorded separately:

```
CREATE TABLE retrieval_scopes (
    retrieval_scope_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    scope_type         STRING NOT NULL,
    scope_key          STRING NOT NULL,
    scope_name         STRING NOT NULL,
    UNIQUE (scope_type, scope_key)
);
```

```
CREATE TABLE document_versions (
    document_version_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    document_id          UUID NOT NULL REFERENCES documents
(document_id),
    retrieval_scope_id   UUID NOT NULL
REFERENCES retrieval_scopes (retrieval_scope_id),
    version_label        STRING NOT NULL,
    source_uri           STRING NOT NULL,
    mime_type            STRING,
    approval_status      STRING,
    effective_at         TIMESTAMPTZ,
    source_modified_at   TIMESTAMPTZ,
    content_hash          STRING NOT NULL,
    ingested_at          TIMESTAMPTZ NOT NULL DEFAULT now(),
    is_current           BOOL NOT NULL DEFAULT true,
    UNIQUE (document_id, version_label),
    UNIQUE (document_version_id, retrieval_scope_id)
);
```

```
CREATE UNIQUE INDEX one_current_version_per_document
ON document_versions (document_id)
WHERE is_current;
```

A retrieval scope defines where a document should be searched.

Examples include:

1. A specific plant.
2. A specific equipment model.
3. A fleet wide engineering standard.
4. A company wide safety procedure.

For the initial design, each document version should have one primary retrieval scope. A model manual belongs to the equipment model scope. A plant procedure belongs to the plant scope. A company standard belongs to the company scope.

The application searches the relevant scopes separately and combines the results.

Document chunks and embeddings

```
CREATE TABLE document_chunks (  
    chunk_id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
    document_version_id  UUID NOT NULL,  
    retrieval_scope_id  UUID NOT NULL,  
    chunk_number       INT NOT NULL,  
    content_kind        STRING NOT NULL DEFAULT 'text',  
    page_number         INT,  
    section_title       STRING,  
    chunk_text          STRING NOT NULL,  
    token_count         INT,  
    embedding           VECTOR(1536) NOT NULL,  
    embedding_model      STRING NOT NULL,  
    embedding_version    STRING NOT NULL,  
    created_at          TIMESTAMPTZ NOT NULL DEFAULT now(),  
  
    FOREIGN KEY (  
        document_version_id,  
        retrieval_scope_id  
    )  
    REFERENCES document_versions (  
        document_version_id,  
        retrieval_scope_id  
    ),  
  
    UNIQUE (document_version_id, chunk_number)  
);
```

`chunk_text` contains the extracted section that will be given to the language model.

`embedding` contains the numerical representation used to find similar sections.

`page_number` and `section_title` support citations.

`content_kind` identifies whether the section came from normal text, a table, an OCR result, or a generated description of a diagram.

`VECTOR(1536)` is only an example. The dimension must match the embedding model selected for the evaluation. CRDB enforces the declared vector dimension. The embedding model and distance function must be selected and tested together. [CockroachDB VECTOR documentation](#)

If the selected model uses a different dimension, the schema must use that dimension. If NextEra later moves to a model with a different dimension, the vectors will need a new column or table and a new index.

Vector index

For the evaluation, the vector index should be scoped by `retrieval_scope_id`:

```
CREATE VECTOR INDEX document_chunks_vector_idx
ON document_chunks (
    retrieval_scope_id,
    embedding
);
```

A scoped search would look like this:

```
SELECT
    chunk_id,
    document_version_id,
    page_number,
    section_title,
    chunk_text,
    embedding <-> $1 AS distance
FROM document_chunks
WHERE retrieval_scope_id = $2
ORDER BY embedding <-> $1
LIMIT 10;
```

This example uses L2 distance. The final distance operator must match the selected embedding model and the vector index configuration.

For this engagement, we should assume filtered vector acceleration depends on matching the vector index prefix columns until we validate the exact behavior on the target CRDB release.

We should confirm the query plan with **EXPLAIN** and test it with NextEra's document volume and query patterns. We should not make a latency or scale commitment before that validation.

Exact document matching without full text search

Vector search is useful for meaning based questions, but fault codes, equipment models, procedure numbers, and part numbers should be matched exactly.

We should store those relationships as normal rows.

```
CREATE TABLE document_fault_code_links (  
    document_version_id  UUID NOT NULL  
    REFERENCES document_versions (document_version_id),  
    fault_code           STRING NOT NULL,  
    PRIMARY KEY (document_version_id, fault_code)  
);
```

```
CREATE INDEX documents_by_fault_code  
ON document_fault_code_links (  
    fault_code,  
    document_version_id  
);
```

```
CREATE TABLE document_equipment_links (  
    document_version_id  UUID NOT NULL  
    REFERENCES document_versions (document_version_id),  
    equipment_id         UUID NOT NULL  
    REFERENCES equipment (equipment_id),  
    relationship_type     STRING NOT NULL,  
    PRIMARY KEY (  
        document_version_id,  
        equipment_id,  
        relationship_type  
    )  
);
```

```
CREATE INDEX documents_by_equipment
```

```
ON document_equipment_links (  
    equipment_id,  
    relationship_type  
);
```

If a user asks about fault code B17, the application first finds documents explicitly linked to B17. It can then run vector search over the relevant document scopes.

The application combines exact relational matches and vector matches. We should not describe this as a native CRDB hybrid search feature.

CRDB supports full text search, but a full text index is an inverted index created using [GIN](#). This design intentionally does not depend on that path. [CockroachDB full text search documentation](#)

If NextEra later requires large scale arbitrary keyword search across all document text, we will need to evaluate that requirement separately. We should not assume that vector search is a complete replacement for every form of keyword search.

Permissions

Permissions should be checked before vector retrieval.

```
CREATE TABLE retrieval_scope_permissions (  
    retrieval_scope_id UUID NOT NULL  
    REFERENCES retrieval_scopes (retrieval_scope_id),  
    principal_type STRING NOT NULL,  
    principal_id STRING NOT NULL,  
    permission STRING NOT NULL,  
    PRIMARY KEY (  
        retrieval_scope_id,  
        principal_type,  
        principal_id,  
        permission  
    )  
);
```

```
CREATE INDEX permissions_by_principal  
ON retrieval_scope_permissions (  
    principal_type,  
    principal_id,  
    retrieval_scope_id
```

```
);
```

The application first determines which scopes the user is allowed to access. It then searches only those scopes.

The source URI does not provide or enforce this permission.

Diagnostic runs

For the first equipment health workflow, use typed diagnostic parameters.

```
CREATE TABLE diagnostic_runs (  
    diagnostic_run_id    UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
    equipment_id         UUID NOT NULL REFERENCES equipment  
(equipment_id),  
    fault_event_id       UUID REFERENCES equipment_fault_events  
(fault_event_id),  
    diagnostic_name       STRING NOT NULL,  
    diagnostic_version    STRING NOT NULL,  
    lookback_hours        INT NOT NULL,  
    comparison_group      STRING,  
    minimum_sample_count  INT,  
    source_window_start   TIMESTAMPTZ,  
    source_window_end     TIMESTAMPTZ,  
    historian_query_ref     STRING,  
    status                STRING NOT NULL,  
    started_at            TIMESTAMPTZ NOT NULL,  
    completed_at          TIMESTAMPTZ,  
    summary               STRING  
);
```

If another diagnostic requires different parameters, add a typed table for that diagnostic. We should not create a generic JSONB parameters field simply to avoid defining the data.

Each finding is stored as a row:

```
CREATE TABLE diagnostic_findings (  
    diagnostic_run_id    UUID NOT NULL  
    REFERENCES diagnostic_runs (diagnostic_run_id),  
    finding_number       INT NOT NULL,  
    check_name           STRING NOT NULL,
```

```

        outcome          STRING NOT NULL,
        observed_value    DECIMAL,
        expected_min      DECIMAL,
        expected_max      DECIMAL,
        unit              STRING,
        explanation       STRING,
        PRIMARY KEY (diagnostic_run_id, finding_number)
);

```

Evidence and citations

Record the document sections and measurements used for each diagnostic:

```

CREATE TABLE diagnostic_document_sources (
    diagnostic_run_id  UUID NOT NULL
    REFERENCES diagnostic_runs (diagnostic_run_id),
    chunk_id          UUID NOT NULL
    REFERENCES document_chunks (chunk_id),
    retrieval_rank     INT,
    vector_distance    FLOAT8,
    PRIMARY KEY (diagnostic_run_id, chunk_id)
);

CREATE TABLE diagnostic_metric_sources (
    diagnostic_run_id  UUID NOT NULL
    REFERENCES diagnostic_runs (diagnostic_run_id),
    metric_summary_id  UUID NOT NULL
    REFERENCES equipment_metric_summaries (metric_summary_id),
    PRIMARY KEY (diagnostic_run_id, metric_summary_id)
);

```

This provides a record of the facts, calculations, and document sections used in the response.

Query flow

For a question such as:

Why did inverter 42 enter a derated state after fault B17?

The application follows this flow:

1. Resolve inverter 42 using the equipment table.
2. Confirm the user can access the equipment and relevant document scopes.
3. Retrieve fault B17 from the fault event table.
4. Retrieve recent typed health summaries.
5. Call the Historian through a diagnostic service if new calculations are required.
6. Store the diagnostic findings in CRDB.
7. Find documents explicitly linked to fault B17, the equipment, or its model.
8. Generate an embedding for the user's question.
9. Search the authorized model, plant, and company document scopes.
10. Retrieve the matching chunk text, document version, page number, and source URI.
11. Give the language model the equipment facts, diagnostic findings, and document sections.
12. Ask the language model to explain the result using only the supplied evidence.
13. Return the answer with document citations and the relevant measurement window.
14. Record the evidence used for the answer.

The language model explains the findings. It does not perform the engineering calculation or invent missing values.

Document refresh flow

When a document changes: One of the approaches, this can be thought through and refined.

1. The connector detects a new version, modification time, or content hash.
2. The processing service retrieves the new approved version.
3. It extracts the content again.
4. It creates new chunks.
5. It generates new embeddings.
6. It writes a new `document_versions` row.
7. It marks the previous version as no longer current.
8. New searches use the current approved version.
9. Older diagnostic records continue to point to the version and chunks that were used at the time.

This is why document versions and content hashes matter. A URI by itself is not enough.

JSONB policy

JSONB should be added only when:

1. We have inspected the actual source payload.
2. The fields genuinely vary and cannot be known in advance.
3. The fields are not needed for joins, permissions, routine filters, or reports.
4. Normal columns or related tables would create unreasonable complexity.

5. The value is small.
6. Keeping the original structure provides a clear benefit.

CRDB supports JSONB. We are choosing not to use it as a default data model. [CockroachDB JSONB documentation](#)

What we need to validate

The evaluation should use one equipment health workflow, such as an inverter fault or battery health investigation.

NextEra should provide:

1. A representative set of approved manuals and procedures.
2. The document permissions that apply to those files.
3. One equipment model.
4. Representative fault events.
5. Selected Historian measurements or access to the required API.
6. Questions that technicians or engineers actually ask.
7. Expected answers reviewed by a subject matter expert.

We should measure:

1. Whether the correct equipment and fault are resolved.
2. Whether exact fault codes and model numbers are matched correctly.
3. Whether vector retrieval returns the correct document sections.
4. Whether every answer cites an approved document version and page.
5. Whether the diagnostic calculations match NextEra's accepted method.
6. Whether permissions are enforced before retrieval.
7. Retrieval and response latency at a representative scale.
8. Vector index size and ingestion behavior.
9. The effect of document updates and embedding regeneration.
10. Whether the proposed retrieval scopes match how NextEra organizes plants, models, and procedures.

The success thresholds should be agreed with NextEra after we inspect the sample data. We should not invent scale or latency commitments before running the evaluation.

Accurate customer positioning

We should say:

CRDB would not replace the Historian or the document repository. It would store and connect the equipment model, fault events, selected health summaries,

approved document sections, vector embeddings, permissions, diagnostic state, and source citations used by the application.

We should also say:

The original manuals remain in SharePoint or object storage. CRDB stores the document catalog, approved versions, extracted sections, embeddings, and links back to the original source.

For grounding:

CRDB does not ground an answer by itself. It stores the trusted facts, diagnostic results, approved document sections, embeddings, permissions, and citations that the application uses to produce a grounded answer.

For the URI:

The URI is a pointer to the original source. It supports verification, citations, document refresh, and audit. It is not the document content, a search index, or a permission mechanism.

For retrieval:

The application combines exact relational matches with scoped vector retrieval. We should not position this as a native built in hybrid search feature.

For product fit:

The value of CRDB is that the application can keep equipment state, faults, calculated health results, document relationships, embeddings, permissions, workflow state, and citations together without introducing another operational database for this part of the application.